aiGym

Service Description

Service ID: *"aiGym"*

Abstract
This document describes the `aiGym` service, version 1.0.0, produced by the AIGym support system of Arrowhead (documentation edition 5.3.0).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Significant Prior Art . . . 3
2.2 How This Service Is Meant to Be Used . . . 3
2.3 Important Delimitations and Dependencies . . . 3

3 Service Interface **4**
3.1 operation status . . . 4
3.2 operation add-sample . . . 4
3.3 operation predict . . . 4
3.4 operation observations . . . 4

4 Information Model **5**
4.1 struct Prediction . . . 5
4.2 struct PredictionRequest . . . 5
4.3 struct Sample . . . 5
4.4 struct SampleRequest . . . 5
4.5 Primitives . . . 5

5 References **7**

6 Revision History **8**
6.1 Amendments . . . 8
6.2 Quality Assurance . . . 8

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `aiGym` service, which provides a small experimentation service for machine learning in the local cloud: collecting training samples and k-nearest-neighbour predictions, fed from the sysMonitor data stream. It is produced by the AIGym system [1] in service version 1.0.0.

The rest of this document is organized as follows. In Section 3, we describe the abstract message operations provided by the service. In Section 4, we end the document by presenting the data types used by the mentioned operations. The concrete interface is specified in the HTTP/TLS/JSON IDD [2].

2.1 Significant Prior Art

New in this workspace.

2.2 How This Service Is Meant to Be Used

Clients add samples (`add-sample`) and ask for predictions (`predict`); `status` and `observations` show the model store and the observations received from `sysMonitorData`. Each sample and inference is also published as a sysMonitor observation (`trainingSample`, `inference`).

2.3 Important Delimitations and Dependencies

Every call is checked with ConsumerAuthorization (`aiGym`, scope = operation). Models are kept in memory.

3 Service Interface

This section describes the interfaces to the `aiGym` service. In particular, each subsection names an abstract operation, an input type and an output type, in that order. The input type is named inside parentheses, while the output type is preceded by a colon. Input and output types are only denoted when accepted or returned, respectively, by the interface in question.

All abstract data types named in this section are defined in Section 4.

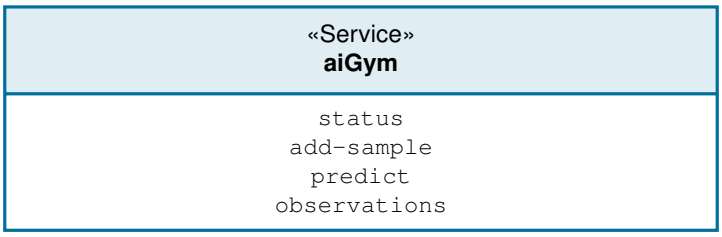


Figure 1: SysML-style block of the `aiGym` service and its operations.

The following interface operations are available.

3.1 operation status () : Map<String, Object>

Returns the state of the model store and the data stream.

3.2 operation add-sample (SampleRequest) : Sample

Adds a training sample to a model.

3.3 operation predict (PredictionRequest) : Prediction

Predicts the target of a feature vector with k nearest neighbours (default 3).

3.4 operation observations () : List<Map<String, Object>>

Returns the latest observations received from `sysMonitorData`.

4 Information Model

Here, all data objects that can be part of the `aiGym` service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is used to denote a collection of named fields, each with its own data type. As a complement to the explicitly defined types in this section, there is also a list of implicit primitive types in Section 4.5, which are used to represent things like hashes and identifiers.

4.1 struct Prediction

Field	Type	Description
model	String	
value	String	
neighbours	Number	
trainingSamples	Number	
createdAt	String	UTC time of creation.

4.2 struct PredictionRequest

Field	Type	Description
model	String	
features	Map<String, String>	
neighbours	Number	

4.3 struct Sample

Field	Type	Description
id	Identifier	Identifier of the record.
model	String	
features	Map<String, String>	
target	String	Target of the policy or token: a service definition or event type.
createdAt	String	UTC time of creation.

4.4 struct SampleRequest

Field	Type	Description
model	String	
features	Map<String, String>	
target	String	Target of the policy or token: a service definition or event type.

4.5 Primitives

Types and structures mentioned throughout this document that are assumed to be available to implementations of this service. The concrete interpretations of each of these types and structures must be provided by any IDD document claiming to implement this service.

Type	Description
Identifier	A string that identifies a record, e.g. a UUID or a composite identifier.
List	An *array* of a known number of items, each having type A.
Map	A set of key-value pairs.
Number	An integer number.
Object	Any structured value.
String	A chain of characters.

5 References

[1] J. Delsing, “AIGym – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-support-java-spring/AIgym/documentation/v5.3.0/AIGym-v5.3.0-SysD.pdf.

[2] ——, “aiGym HTTP/TLS/JSON – Interface Design Description (AIGym),” Eclipse Arrowhead, IDD 5.3.0, 2026, aIGym-v5.3.0-aiGym-HTTP-IDD.pdf.

6 Revision History

6.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)
3	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

6.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	